



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)
دانشکده مهندسی کامپیوتر

پایان نامه کارشناسی

طراحی سامانه ثبت وقایع برای تحلیل بی درنگ کارایی فرایندهای JVM.

نگارش

محمد امین صدرنشین

استاد راهنما

دکتر حامد فریه

استاد مشاور

دکتر سید احمد جوادی

اسفند ۱۴۰۴

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

صفحه فرم ارزیابی و تصویب پایان نامه - فرم تأیید اعضاء کمیته دفاع

در این صفحه فرم دفاع یا تأیید و تصویب پایان نامه موسوم به فرم کمیته دفاع - موجود در پرونده آموزشی - را قرار دهید.

نکات مهم:

- نگارش پایان نامه/رساله باید به **زبان فارسی** و بر اساس آخرین نسخه دستورالعمل و راهنمای تدوین پایان نامه های دانشگاه صنعتی امیرکبیر باشد.(دستورالعمل و راهنمای حاضر)
- رنگ جلد پایان نامه/رساله چاپی کارشناسی، کارشناسی ارشد و دکترا باید به ترتیب مشکی، طوسی و سفید رنگ باشد.
- چاپ و صحافی پایان نامه/رساله بصورت **پشت و رو(دورو)** بلامانع است و انجام آن توصیه می شود.



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

به نام خدا

تعهدنامه اصالت اثر

تاریخ: اسفند ۱۴۰۴

اینجانب **محمد امین صدرنشین** متعهد می‌شوم که مطالب مندرج در این پایان‌نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی اساتید دانشگاه صنعتی امیرکبیر بوده و به دستاوردهای دیگران که در این پژوهش از آنها استفاده شده است مطابق مقررات و روال متعارف ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان‌نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نگردیده است.

در صورت اثبات تخلف در هر زمان، مدرک تحصیلی صادر شده توسط دانشگاه از درجه اعتبار ساقط بوده و دانشگاه حق پیگیری قانونی خواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان‌نامه متعلق به دانشگاه صنعتی امیرکبیر می‌باشد. هرگونه استفاده از نتایج علمی و عملی، واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان‌نامه بدون موافقت کتبی دانشگاه صنعتی امیرکبیر ممنوع است. نقل مطالب با ذکر مآخذ بلامانع است.

محمد امین صدرنشین

امضا

نویسنده پایان نامه، در صورت تمایل میتواند برای پاسخگویی پایان نامه خود را به شخص
یا اشخاص و یا ارگان خاصی تقدیم نماید.

سپاسگزاری

نویسنده پایان نامه می تواند مراتب امتنان خود را نسبت به استاد راهنما و استاد مشاور و یا دیگر افرادی که طی انجام پایان نامه به نحوی او را یاری و یا با او همکاری نموده اند ابراز دارد.

محمد امین صدر نشین
اسفند ۱۴۰۴

چکیده

افزایش پیچیدگی سامانه‌های نرم‌افزاری و اجرای پیوسته آن‌ها در محیط‌های توزیع‌شده، نیاز به پایش دقیق، کم‌هزینه و قابل تحلیل را پررنگ کرده است. در این پایان‌نامه سامانه‌ای برای جمع‌آوری، جمع‌آوری و تحلیل رخدادهای عملکردی برنامه‌های مبتنی بر ماشین مجازی جاوا طراحی و پیاده‌سازی شده است. سامانه پیشنهادی از Java Flight Recorder برای دریافت رخدادهای سطح برنامه، از eBPF برای مشاهده رخدادهای سطح هسته، از Kafka برای انتقال جریان داده، از Kafka Streams برای جمع‌پنجره‌ای، و از یک بخش تحلیلی پایتونی برای تشخیص ناهنجاری‌ها استفاده می‌کند. معماری سامانه به صورت خط لوله‌ای طراحی شده است تا دریافت رخداد، جمع‌آوری، تحلیل و تولید گزارش از یکدیگر جدا باشند. در بخش دریافت، رخدادهای JFR و رخدادهای هسته به پیام‌های ساخت‌یافته تبدیل می‌شوند. سپس رخدادها در بازه‌های زمانی قابل تنظیم جمع‌آوری شده و معیارهایی مانند مصرف پردازنده، حافظه، ورودی/خروجی فایل، شبکه، قفل‌ها و شمارنده‌های سیستمی استخراج می‌شود. در مرحله تحلیل، آشکارسازهای مبتنی بر آستانه، خط پایه غلتان و روند افزایشی، گزارش ناهنجاری‌ها را تولید می‌کنند. برای ارزیابی، مجموعه‌ای از سناریوهای کنترل‌شده شامل جهش پردازنده، فشار حافظه و جمع‌آوری زباله، انفجار ورودی/خروجی فایل، ازدحام قفل، ترافیک شبکه و دسترسی حافظه‌نگاشته به فایل در مخزن پروژه پیاده‌سازی شده است. نتایج نشان می‌دهد که جداسازی اجزای خط لوله، توسعه‌پذیری سامانه را افزایش داده و امکان تحلیل همزمان رخدادهای سطح برنامه و سطح سیستم‌عامل را فراهم می‌کند.

واژه‌های کلیدی:

پایش عملکرد، تحلیل بی‌درنگ، ثبت وقایع، JVM، Java Flight Recorder

فهرست مطالب

صفحه

عنوان

۱	فهرست نمادها و کوتاه‌نوشت‌ها	۱
۲	۱ کلیات پژوهش	۲
۳	۱-۱ مقدمه	۳
۳	۲-۱ بیان مسئله	۳
۵	۳-۱ اهمیت و ضرورت پژوهش	۵
۶	۴-۱ روش انجام پژوهش	۶
۷	۵-۱ محدوده و محدودیت‌های پژوهش	۷
۹	۶-۱ ساختار پایان‌نامه	۹
۱۰	۷-۱ جمع‌بندی	۱۰
۱۱	۲ مبانی نظری و فناوری‌های مورد استفاده	۱۱
۱۲	۱-۲ مقدمه	۱۲
۱۲	۲-۲ پایش، مشاهده‌پذیری و پروفایل‌سازی	۱۲
۱۲	۱-۲-۲ پایش عملکرد	۱۲
۱۳	۲-۲-۲ مشاهده‌پذیری	۱۳
۱۳	۳-۲-۲ پروفایل‌سازی	۱۳
۱۴	۴-۲-۲ رخداد، معیار و گزارش	۱۴
۱۴	۳-۲ اجرای برنامه‌های جاوا و ماشین مجازی جاوا	۱۴
۱۴	۱-۳-۲ کامپایل درجا	۱۴
۱۵	۲-۳-۲ مدیریت حافظه و جمع‌آوری زباله	۱۵
۱۵	۳-۳-۲ نخ‌ها و هم‌زمانی	۱۵
۱۵	۴-۲ Java Flight Recorder	۱۵
۱۵	۱-۴-۲ مدل رخداد در JFR	۱۵
۱۶	۲-۴-۲ دسته‌های مهم رخدادها در JFR	۱۶
۱۶	۳-۴-۲ ثبت پیوسته و مصرف جریانی	۱۶

۱۶	۴-۴-۲ مزایای JFR
۱۷	۵-۴-۲ محدودیت‌های JFR
۱۷	۵-۲ فناوری eBPF و مشاهده رخدادهای سطح هسته
۱۷	۱-۵-۲ نحوه اجرای برنامه‌های eBPF
۱۸	۲-۵-۲ نقاط اتصال
۱۸	۳-۵-۲ نقشه‌ها
۱۹	۴-۵-۲ بافر حلقوی
۱۹	۵-۵-۲ شناسه فرایند و نخ
۱۹	۶-۵-۲ مزایای eBPF
۲۰	۷-۵-۲ محدودیت‌ها و چالش‌های eBPF
۲۰	۶-۲ ترکیب داده‌های JVM و سیستم‌عامل
۲۱	۷-۲ انتقال و پردازش جریانی داده با Apache Kafka
۲۲	۸-۲ تشخیص ناهنجاری در داده‌های پایشی
۲۳	۱-۸-۲ روش آستانه‌ای
۲۳	۲-۸-۲ خط پایه غلتان
۲۳	۳-۸-۲ تشخیص روند
۲۴	۴-۸-۲ معیارهای ارزیابی آشکارساز
۲۴	۹-۲ ملاحظات سربار و دقت
۲۵	۱۰-۲ جمع‌بندی
۲۶	۳ معماری سامانه
۲۷	۱-۳ مقدمه
۲۷	۲-۳ نمای کلی معماری
۲۸	۳-۳ مسیرهای داده در سامانه
۲۸	۱-۳-۳ مسیر رخدادهای JFR
۲۸	۲-۳-۳ مسیر رخدادهای هسته
۲۹	۴-۳ لایه جمع‌آوری داده
۲۹	۵-۳ لایه انتقال و ذخیره‌سازی جریان داده

۲۹	۶-۳ لایه تجميع
۳۰	۷-۳ لایه تحليل
۳۱	۸-۳ لایه ارزیابی
۳۱	۹-۳ مدل داده مشترک
۳۱	۱۰-۳ دلایل انتخاب معماری لایه‌ای
۳۲	۱۱-۳ جمع‌بندی
۳۳	۴ پیاده‌سازی سامانه
۳۴	۱-۴ ساختار مخزن
۳۴	۲-۴ دریافت رخدادهای JFR
۳۴	۳-۴ دریافت رخدادهای هسته
۳۴	۴-۴ تجميع رخدادها
۳۵	۵-۴ تحليل و گزارش‌گیری
۳۵	۶-۴ پایش خود سامانه
۳۵	۷-۴ جمع‌بندی
۳۶	۵ ارزیابی، نتیجه‌گیری و کارهای آینده
۳۷	۱-۵ روش ارزیابی
۳۷	۲-۵ سناریوهای آزمایش
۳۷	۳-۵ معیارهای ارزیابی
۳۸	۴-۵ بحث
۳۸	۵-۵ نتیجه‌گیری
۳۸	۶-۵ کارهای آینده
۴۰	کتاب‌نامه
۴۱	آ راهنمای اجرای پروژه
۴۲	۱-آ ساخت ماژول‌های جاوا
۴۲	۲-آ اجرای اجزای اصلی
۴۲	۳-آ اجرای ارزیابی

۴۲		آ-۴ خروجی‌ها
۴۴		واژه‌نامه‌ی فارسی به انگلیسی
۴۶		واژه‌نامه‌ی انگلیسی به فارسی

فهرست تصاویر

صفحه

شکل

۲۷ ۱-۳ نمای کلی معماری لایه‌ای سامانه

فهرست جداول

صفحه

جدول

فهرست نمادها و کوتاه‌نوشت‌ها

JFR	Java Flight Recorder، ابزار ثبت رخدادهای زمان اجرای جاوا
JVM	Java Virtual Machine، ماشین مجازی جاوا
eBPF	extended Berkeley Packet Filter، چارچوب اجرای برنامه‌های کنترل‌شده در هسته لینوکس
Kafka	سکوی پیام‌رسانی و جریان داده توزیع‌شده
CSV	قالب متنی جدولی برای ذخیره گزارش‌ها
GC	Garbage Collection، فرایند آزادسازی حافظه در جاوا
PID	شناسه فرایند در سیستم‌عامل

فصل اول

کلیات پژوهش

۱-۱ مقدمه

با افزایش پیچیدگی سامانه‌های نرم‌افزاری، تحلیل رفتار برنامه‌ها در زمان اجرا به یکی از نیازهای مهم در فرایند توسعه، نگهداری و بهره‌برداری از نرم‌افزار تبدیل شده است. برنامه‌های مبتنی بر ماشین مجازی جاوا ممکن است به صورت هم‌زمان تحت تأثیر عواملی مانند مدیریت خودکار حافظه، زمان‌بندی نخ‌ها، عملیات ورودی و خروجی، ارتباطات شبکه‌ای، رقابت بر سر قفل‌ها و رفتار سیستم‌عامل قرار گیرند. از این رو، بررسی تنها یک لایه از سامانه معمولاً برای شناسایی دقیق مشکلات عملکردی کافی نیست.

ابزارهای پایش سطح ماشین مجازی، اطلاعات ارزشمندی درباره رفتار داخلی برنامه ارائه می‌کنند، اما دید کاملی از فعالیت‌های سطح سیستم‌عامل ندارند. در مقابل، ابزارهای سطح سیستم‌عامل قادر به مشاهده مصرف منابع و رخداد‌های هسته هستند، ولی ارتباط آن‌ها با مفاهیم سطح برنامه و ماشین مجازی همیشه به سادگی قابل تشخیص نیست. ترکیب اطلاعات این دو سطح می‌تواند دید جامع‌تری از رفتار برنامه فراهم کند و فرایند شناسایی گلوگاه‌ها و ناهنجاری‌های عملکردی را بهبود بخشد.

در این پژوهش، سامانه‌ای برای جمع‌آوری، انتقال، تجمیع و تحلیل رخداد‌های برنامه‌های جاوا طراحی و پیاده‌سازی شده است. این سامانه با استفاده از داده‌های سطح ماشین مجازی و سیستم‌عامل، شاخص‌های عملکردی را استخراج کرده و از آن‌ها برای تحلیل رفتار برنامه و تشخیص ناهنجاری استفاده می‌کند. در ادامه این فصل، مسئله پژوهش، اهمیت آن، اهداف، پرسش‌ها، روش انجام کار، محدوده پژوهش و ساختار پایان‌نامه تشریح می‌شوند.

۲-۱ بیان مسئله

عملکرد یک برنامه جاوا تنها به منطق پیاده‌سازی شده در کد برنامه وابسته نیست. نحوه اجرای کد توسط ماشین مجازی جاوا، فعالیت کامپایلر در جا^۱، مدیریت حافظه و جمع‌آوری زباله، زمان‌بندی نخ‌ها، عملیات ورودی و خروجی و تعامل برنامه با هسته سیستم‌عامل، همگی بر رفتار نهایی برنامه اثر می‌گذارند. به همین دلیل، مشاهده یک معیار کلی مانند درصد مصرف پردازنده یا میزان حافظه مصرف‌شده، به تنهایی برای تشخیص علت افت عملکرد کافی نیست.

برای نمونه، افزایش زمان پاسخ یک برنامه می‌تواند ناشی از اجرای محاسبات سنگین، انتظار برای دریافت داده از شبکه، کندی عملیات فایل، توقف‌های جمع‌آوری زباله، رقابت چند نخ بر سر یک قفل

^۱Just-in-Time Compiler

یا تأخیر ناشی از زمان‌بندی سیستم‌عامل باشد. این وضعیت‌ها ممکن است از دید کاربر نهایی نتیجه مشابهی داشته باشند، اما علت و راهکار رفع آن‌ها متفاوت است. بنابراین، سامانه پایش باید بتواند علاوه بر اندازه‌گیری مصرف منابع، زمینه وقوع رخداد و ارتباط آن با فعالیت‌های ماشین مجازی و سیستم‌عامل را نیز تا حد امکان حفظ کند.

روش‌های متداول پایش را می‌توان به‌طور کلی در دو گروه قرار داد. گروه نخست، ابزارهایی هستند که در سطح برنامه یا ماشین مجازی فعالیت می‌کنند و اطلاعاتی مانند وضعیت نخ‌ها، تخصیص حافظه، رخدادهای جمع‌آوری زباله، قفل‌ها و نمونه‌های پشته اجرا را در اختیار قرار می‌دهند. این ابزارها دید مناسبی از رفتار داخلی برنامه دارند، اما برخی رخدادهای سطح پایین سیستم‌عامل را مشاهده نمی‌کنند. گروه دوم، ابزارهای پایش سیستم‌عامل هستند که اطلاعاتی درباره پردازنده، حافظه، فایل، شبکه، فراخوانی‌های سیستمی و زمان‌بندی پردازش‌ها فراهم می‌کنند. این ابزارها معمولاً دید مستقیمی از ساختار سطح بالای برنامه جاوا و رخدادهای داخلی ماشین مجازی ندارند.

جدایی میان این دو سطح مشاهده، فرایند ریشه‌یابی مشکلات را دشوار می‌کند. برای مثال، ابزار سطح سیستم‌عامل ممکن است افزایش عملیات خواندن فایل را نشان دهد، اما مشخص نکند این فعالیت در چه زمینه‌ای از اجرای برنامه رخ داده است. در مقابل، ابزار سطح ماشین مجازی ممکن است توقف یا انتظار یک نخ را ثبت کند، اما علت سطح پایین آن را به‌طور کامل نمایش ندهد. در نتیجه، نیاز به رویکردی وجود دارد که داده‌های هر دو سطح را در یک خط لوله واحد جمع‌آوری و پردازش کند.

چالش دیگر، حجم و نرخ تولید رخدادهای است. یک برنامه در حال اجرا می‌تواند در مدت کوتاهی تعداد زیادی رخداد مربوط به نخ‌ها، حافظه، فایل، شبکه و هسته تولید کند. نگهداری و تحلیل مستقیم همه رخدادهای خام، هم هزینه ذخیره‌سازی را افزایش می‌دهد و هم تحلیل سریع داده‌ها را دشوار می‌کند. به همین دلیل، لازم است رخدادهای بازه‌های زمانی مشخص تجمیع شوند و به شاخص‌هایی مانند تعداد رخداد، مجموع زمان، میانگین، بیشینه، نرخ عملیات و حجم داده تبدیل شوند. این شاخص‌ها ضمن کاهش حجم داده، امکان مقایسه وضعیت جاری با رفتار عادی سامانه را فراهم می‌کنند.

در کنار جمع‌آوری و تجمیع داده، تشخیص ناهنجاری نیز مسئله‌ای اساسی است. صرف نمایش مقادیر خام یا نمودارها، مسئولیت تفسیر تمام داده‌ها را بر عهده کاربر قرار می‌دهد. در سامانه‌هایی که به‌صورت پیوسته اجرا می‌شوند، بررسی دستی همه رخدادهای عملی نیست. بنابراین، سامانه باید بتواند بر اساس قواعد قابل توضیح و قابل توسعه، رفتارهای غیرعادی را شناسایی کرده و اطلاعات لازم برای بررسی دقیق‌تر را در قالب گزارش ارائه دهد.

بر این اساس، مسئله اصلی این پژوهش طراحی و پیاده‌سازی یک خط لوله پایش عملکرد برای

برنامه‌های جاوا است که بتواند:

- رخدادهای سطح ماشین مجازی جاوا را با سربر قابل قبول جمع‌آوری کند؛
- رخدادهای مرتبط سطح هسته لینوکس را بدون تغییر کد برنامه هدف دریافت کند؛
- داده‌های جمع‌آوری‌شده را از طریق یک زیرساخت پیام‌رسانی جریانی منتقل کند؛
- رخدادهای خام را در پنجره‌های زمانی مشخص به شاخص‌های قابل تحلیل تبدیل کند؛
- ناهنجاری‌های مرتبط با پردازنده، حافظه، فایل، شبکه، قفل‌ها و رخدادهای هسته را تشخیص دهد؛
- و خروجی قابل استفاده‌ای برای تحلیل و ارزیابی رفتار برنامه تولید کند.

پروژه پیاده‌سازی‌شده در این پژوهش از Java Flight Recorder یا JFR برای دریافت رخدادهای درون ماشین مجازی، از eBPF برای مشاهده رخدادهای سطح هسته لینوکس، از Apache Kafka برای انتقال جریان داده، از Kafka Streams برای تجمیع پنجره‌ای و از یک مؤلفه تحلیلی برای تشخیص و گزارش ناهنجاری‌ها استفاده می‌کند [۲، ۱، ۴، ۵، ۳].

۳-۱ اهمیت و ضرورت پژوهش

نیاز به پایش عملکرد تنها به مرحله توسعه نرم‌افزار محدود نیست. بسیاری از مشکلات زمانی آشکار می‌شوند که برنامه تحت بار واقعی، روی سخت‌افزار مشخص و در تعامل با سرویس‌ها و منابع خارجی اجرا شود. برخی از این مشکلات نیز ماهیتی موقتی دارند و تنها در بازه کوتاهی رخ می‌دهند. در چنین شرایطی، بازآفرینی مشکل در محیط آزمایشگاهی ممکن است دشوار یا حتی غیرممکن باشد. سامانه پایش زمان اجرا می‌تواند شواهد لازم برای تحلیل این وضعیت‌ها را جمع‌آوری و حفظ کند.

یکی از ضرورت‌های مهم، کاهش زمان تشخیص علت مشکل است. هنگامی که اطلاعات سطح ماشین مجازی و سیستم‌عامل جدا از یکدیگر جمع‌آوری شوند، تحلیل‌گر ناچار است خروجی چند ابزار را به‌صورت دستی با هم مقایسه کند. اختلاف قالب داده، دقت زمانی متفاوت و نبود شناسه‌های مشترک، این فرایند را پیچیده می‌کند. یک خط لوله یکپارچه می‌تواند رخدادهای را به قالب‌های مشخص تبدیل کرده و تحلیل هم‌زمان آن‌ها را ساده‌تر سازد.

ضرورت دیگر، کنترل سربار پایش است. ابزاری که برای مشاهده رفتار برنامه استفاده می‌شود نباید خود به عامل اصلی افت عملکرد تبدیل شود. استفاده از JFR به عنوان سازوکار ثبت رخداد در سطح ماشین مجازی و eBPF برای مشاهده کنترل شده رخدادهای هسته، امکان جمع‌آوری داده بدون ابزارگذاری گسترده کد برنامه را فراهم می‌کند [۴، ۵]. با این حال، میزان رخدادهای فعال، نرخ نمونه‌برداری، حجم پیام‌ها و نوع پردازش باید به گونه‌ای انتخاب شوند که هزینه پایش قابل کنترل باقی بماند.

پردازش جریانی نیز در این پژوهش اهمیت دارد. در رویکردهای مبتنی بر تحلیل آفلاین، ابتدا حجم زیادی از داده ذخیره و سپس بررسی می‌شود. این روش برای تحلیل تاریخی مفید است، اما در شناسایی سریع وضعیت‌های غیرعادی محدودیت دارد. انتقال رخدادهای از طریق Kafka و تجمیع آن‌ها با Kafka Streams امکان پردازش پیوسته و جداسازی مؤلفه‌های تولید، انتقال و تحلیل را فراهم می‌کند [۱، ۲].

از دید مهندسی نرم‌افزار، توسعه‌پذیری نیز اهمیت دارد. انواع رخدادهای نیازهای پایش در پروژه‌های مختلف یکسان نیستند. سامانه باید به گونه‌ای طراحی شود که افزودن یک نوع رخداد، شاخص جدید یا قانون تشخیص ناهنجاری، نیازمند بازنویسی کامل خط لوله نباشد. جداسازی مؤلفه‌های دریافت رخداد، انتقال، تجمیع، تحلیل و ارزیابی، امکان تغییر و توسعه مستقل هر بخش را افزایش می‌دهد.

در نهایت، این پژوهش تنها به مطالعه نظری ابزارهای پایش محدود نیست، بلکه یک نمونه عملی انتهابه‌انتها را ارائه می‌کند. وجود پیاده‌سازی اجرایی، سناریوهای تولید بار و مؤلفه ارزیابی، امکان بررسی عملی تصمیم‌های معماری و محدودیت‌های راهکار را فراهم می‌سازد.

۴-۱ روش انجام پژوهش

این پژوهش از نوع کاربردی و مبتنی بر طراحی و پیاده‌سازی یک سامانه نرم‌افزاری است. فرایند انجام پژوهش در چند مرحله اصلی صورت گرفته است.

در مرحله نخست، نیازمندی‌های پایش برنامه‌های جاوا و منابع تولید داده بررسی شدند. در این مرحله، رخدادهای قابل دریافت از ماشین مجازی، اطلاعات قابل مشاهده در سطح سیستم عامل و محدودیت‌های روش‌های مختلف جمع‌آوری داده مطالعه شدند. بر اساس این بررسی، JFR برای رخدادهای سطح ماشین مجازی و eBPF برای رخدادهای منتخب سطح هسته انتخاب شدند.

در مرحله دوم، معماری خط لوله سامانه طراحی شد. در این معماری، وظایف جمع‌آوری رخداد، انتقال پیام، تجمیع، تحلیل و ارزیابی در مؤلفه‌های جداگانه قرار گرفتند. جداسازی این وظایف باعث می‌شود هر مؤلفه بتواند مستقل از بخش‌های دیگر توسعه، آزمایش یا جایگزین شود.

در مرحله سوم، مسیر دریافت رخدادهای JFR پیاده‌سازی شد. این بخش مسئول دریافت رخدادهای ماشین مجازی، استخراج ویژگی‌های مورد نیاز و تبدیل آن‌ها به پیام‌های قابل انتشار در زیرساخت جریان است. رخدادهای مورد توجه شامل داده‌های مرتبط با پردازنده، حافظه، جمع‌آوری زباله، نخ‌ها، قفل‌ها، فایل و شبکه هستند.

در مرحله چهارم، مسیر رخدادهای سطح هسته با استفاده از eBPF پیاده‌سازی شد. برنامه‌های eBPF رخدادهای منتخب را در هسته مشاهده کرده و اطلاعات لازم را از طریق سازوکار ارتباطی مناسب به فضای کاربر منتقل می‌کنند. سپس مؤلفه دریافت‌کننده، این داده‌ها را به پیام‌های جریان پایش تبدیل می‌کند.

در مرحله پنجم، Kafka به‌عنوان لایه انتقال و جداسازی مؤلفه‌ها به کار گرفته شد. رخدادهای خام در جریان‌های مشخص منتشر می‌شوند و مصرف‌کنندگان می‌توانند بدون وابستگی مستقیم به تولیدکننده، آن‌ها را پردازش کنند. این ساختار امکان گسترش مستقل بخش‌های دریافت، تجمیع و تحلیل را فراهم می‌سازد.

در مرحله ششم، رخدادهای خام با استفاده از Kafka Streams در پنجره‌های زمانی تجمیع شدند. هدف این مرحله، تبدیل تعداد زیادی رخداد جزئی به شاخص‌هایی فشرده‌تر و قابل مقایسه بود. بسته به نوع رخداد، مقادیری مانند تعداد، مجموع، میانگین، بیشینه، نرخ و حجم داده محاسبه می‌شوند.

در مرحله هفتم، مؤلفه تحلیل و تشخیص ناهنجاری توسعه یافت. در این مؤلفه، قواعدی برای بررسی شاخص‌های تجمیع‌شده تعریف می‌شوند. این قواعد می‌توانند بر اساس آستانه‌های مشخص، مقایسه با خط پایه نزدیک یا بررسی روند تغییرات عمل کنند. انتخاب روش‌های قابل توضیح باعث می‌شود علت تولید هر هشدار برای کاربر قابل درک باشد.

در مرحله نهم، سناریوهای ارزیابی برای ایجاد فشار پردازنده، مصرف حافظه، فعالیت فایل، ارتباط شبکه‌ای، رقابت قفل و رخدادهای سطح هسته اجرا می‌شوند. خروجی سامانه در این سناریوها بررسی شده و علاوه بر توان تشخیص رفتارهای غیرعادی، سربار اجرای مؤلفه‌های پایش نیز اندازه‌گیری می‌شود.

۵-۱ محدوده و محدودیت‌های پژوهش

تعیین محدوده پژوهش برای جلوگیری از برداشت نادرست از اهداف و نتایج ضروری است. سامانه این پژوهش برای پایش برنامه‌های جاوا در محیط لینوکس طراحی شده است. تمرکز اصلی آن بر جمع‌آوری رخدادهای سطح JVM و هسته، پردازش جریانی، استخراج شاخص‌های عملکردی و تشخیص

ناهنجاری‌های مرتبط با منابع است.

موارد زیر در محدوده این پژوهش قرار دارند:

- برنامه‌های اجراشده روی ماشین مجازی جاوا؛
- محیط‌های مبتنی بر سیستم‌عامل لینوکس؛
- رخدادهای منتخب JFR و eBPF؛
- پردازش پیوسته و تجمیع پنجره‌ای رخدادهای؛
- تحلیل پردازنده، حافظه، فایل، شبکه، قفل‌ها و رخدادهای هسته؛
- تشخیص ناهنجاری با روش‌های قاعده‌محور و آماری قابل توضیح؛
- تولید گزارش برای کمک به تحلیل رفتار برنامه؛
- ارزیابی سامانه با بارهای کنترل‌شده.

موارد زیر خارج از محدوده اصلی پژوهش هستند:

- اصلاح یا بهینه‌سازی خودکار کد برنامه پس از تشخیص مشکل؛
- تضمین تشخیص همه انواع خطاها و مشکلات عملکردی؛
- ردیابی توزیع‌شده کامل درخواست‌ها میان چندین سرویس مستقل؛
- تحلیل امنیتی، تشخیص نفوذ یا شناسایی بدافزار؛
- استفاده از مدل‌های پیچیده یادگیری عمیق برای تشخیص ناهنجاری؛
- پشتیبانی عمومی از همه سیستم‌عامل‌ها و همه نسخه‌های ماشین مجازی؛
- انتساب قطعی تمام رخدادهای سطح هسته به یک متد مشخص جاوا.

یکی از محدودیت‌های مهم، تفاوت سطح انتزاع داده‌ها است. رخدادهای JFR به مفاهیم ماشین مجازی و برنامه نزدیک هستند، در حالی که رخدادهای eBPF در سطح فرایند، نخ، فراخوانی سیستمی و

هسته تولید می‌شوند. ایجاد ارتباط دقیق میان این دو سطح همیشه ممکن نیست و به وجود شناسه‌های مشترک و دقت زمانی مناسب وابسته است.

محدودیت دیگر به تنظیمات جمع‌آوری داده مربوط می‌شود. فعال‌سازی تعداد زیاد رخدادها یا انتخاب نرخ نمونه‌برداری بالا می‌تواند حجم پیام و سربار سامانه را افزایش دهد. در مقابل، کاهش بیش از حد نرخ جمع‌آوری ممکن است برخی رخدادها را کوتاه‌مدت را پنهان کند. بنابراین، میان دقت مشاهده و هزینه پایش نوعی مصالحه وجود دارد.

همچنین، نتیجه تشخیص ناهنجاری به انتخاب آستانه‌ها، طول پنجره، خط پایه و نوع بار کاری وابسته است. یک مقدار که برای یک برنامه غیرعادی تلقی می‌شود، ممکن است در برنامه‌ای دیگر کاملاً طبیعی باشد. از این رو، قواعد تشخیص باید با توجه به ویژگی‌های محیط هدف تنظیم شوند و خروجی سامانه به‌عنوان شواهدی برای تحلیل در نظر گرفته شود، نه حکم قطعی درباره علت مشکل.

۶-۱ ساختار پایان‌نامه

این پایان‌نامه در پنج فصل تنظیم شده است. در فصل حاضر، مسئله پژوهش، اهمیت و ضرورت آن، روش انجام کار و محدوده پژوهش بیان شد.

در فصل دوم، مبانی نظری و فناوری‌های مورد استفاده بررسی می‌شوند. مفاهیم پایش، مشاهده‌پذیری و پروفایل‌سازی معرفی شده و سپس JFR، eBPF، پردازش جریانی، Kafka، Kafka Streams و روش‌های تشخیص ناهنجاری تشریح می‌شوند.

فصل سوم به معماری سامانه اختصاص دارد. در این فصل، مؤلفه‌های اصلی، مسیرهای جریان داده، مدل رخدادها، نحوه ارتباط بخش‌های مختلف و تصمیم‌های معماری توضیح داده می‌شوند.

در فصل چهارم، جزئیات پیاده‌سازی ارائه می‌شود. ساختار ماژول‌های پروژه، نحوه دریافت رخدادها، ماشین مجازی و هسته، انتشار پیام‌ها، تجمیع پنجره‌ای، آشکارسازهای ناهنجاری و سازوکار گزارش‌گیری بررسی می‌شوند.

در فصل پنجم، روش و سناریوهای ارزیابی، نتایج حاصل از آزمایش‌ها، سربار سامانه و محدودیت‌های مشاهده‌شده ارائه می‌شوند. در پایان این فصل نیز جمع‌بندی نهایی و پیشنهادهایی برای توسعه آینده مطرح می‌گردند.

۷-۱ جمع‌بندی

در این فصل، ضرورت ایجاد یک دید چندلایه برای پایش برنامه‌های جاوا بیان شد. مشخص شد که اطلاعات سطح ماشین مجازی یا سیستم‌عامل به تنهایی نمی‌توانند تمام ابعاد رفتار برنامه را نشان دهند و ترکیب این دو منبع می‌تواند تحلیل گلوگاه‌ها و رفتارهای غیرعادی را بهبود دهد. مسئله اصلی پژوهش، ایجاد یک خط لوله عملی برای جمع‌آوری، انتقال، تجمیع و تحلیل این رخدادها و تشخیص ناهنجاری‌های عملکردی است.

همچنین روش انجام کار و مرزهای سامانه مشخص شدند. در فصل بعد، مفاهیم و فناوری‌های لازم برای درک معماری و پیاده‌سازی این سامانه بررسی خواهند شد.

فصل دوم

مبانی نظری و فناوری‌های مورد استفاده

۱-۲ مقدمه

برای تحلیل سامانه پایش ارائه شده در این پایان نامه، آشنایی با چند مفهوم و فناوری پایه ضروری است. این سامانه بر ترکیب داده‌های سطح ماشین مجازی جاوا، رخدادهای سطح سیستم عامل، انتقال جریانی داده‌ها، تجمیع پنجره‌ای و تحلیل ناهنجاری‌ها تکیه دارد. بنابراین، پیش از معرفی معماری و جزئیات پیاده‌سازی، لازم است مفاهیم مرتبط با پایش، مشاهده‌پذیری، پروفایل‌سازی، ثبت رخدادهای جاوا، ردیابی سطح هسته و تشخیص ناهنجاری بررسی شوند.

در این فصل ابتدا مفاهیم پایش، مشاهده‌پذیری و پروفایل‌سازی توضیح داده می‌شوند. سپس ابزار Java Flight Recorder یا JFR به عنوان منبع داده‌های سطح ماشین مجازی و فناوری eBPF به عنوان روشی برای مشاهده رخدادهای سطح هسته معرفی می‌شوند. در ادامه، مفاهیم انتقال و پردازش جریانی داده با استفاده از Apache Kafka و Kafka Streams و سپس روش‌های پایه تشخیص ناهنجاری بیان می‌گردند. هدف این فصل، فراهم کردن زمینه نظری لازم برای درک معماری سامانه و جزئیات پیاده‌سازی است.

۲-۲ پایش، مشاهده‌پذیری و پروفایل‌سازی

۱-۲-۲ پایش عملکرد

پایش^۱ به فرایند جمع‌آوری پیوسته اطلاعات درباره وضعیت یک سامانه و بررسی آن بر اساس مجموعه‌ای از شاخص‌های از پیش تعیین شده گفته می‌شود. هدف پایش آن است که وضعیت جاری سامانه مشخص شود و در صورت عبور یک شاخص از محدوده قابل قبول، هشدار مناسبی ایجاد گردد. معیارهایی مانند درصد استفاده از پردازنده، مقدار حافظه مصرف شده، تعداد درخواست‌ها، نرخ خطا و زمان پاسخ از نمونه‌های رایج داده‌های پایشی هستند.

پایش معمولاً بر پرسش‌های مشخصی تمرکز دارد؛ برای مثال، آیا مصرف پردازنده از یک حد معین بیشتر شده است، آیا نرخ خطا افزایش یافته است یا آیا فضای دیسک در حال اتمام است. در نتیجه، پایش برای شناسایی وضعیت‌های شناخته شده بسیار مناسب است، اما ممکن است برای تحلیل مشکلاتی که علت آن‌ها از پیش مشخص نیست کافی نباشد.

¹Monitoring

۲-۲-۲ مشاهده‌پذیری

مشاهده‌پذیری^۲ به توانایی استنتاج وضعیت داخلی سامانه از طریق خروجی‌های قابل مشاهده آن گفته می‌شود. در مهندسی نرم‌افزار، این خروجی‌ها معمولاً شامل معیارها، گزارش‌های متنی و ردهای توزیع شده هستند. رخدادهای زمان اجرا و داده‌های پروفایل‌سازی نیز می‌توانند بخشی از اطلاعات مشاهده‌پذیری باشند.

تفاوت اصلی پایش و مشاهده‌پذیری در نوع پرسش‌هایی است که پاسخ می‌دهند. پایش بیشتر برای پاسخ‌گویی به پرسش‌های از قبل شناخته‌شده به کار می‌رود، در حالی که مشاهده‌پذیری باید امکان بررسی پرسش‌های جدید و پیش‌بینی‌نشده را نیز فراهم کند. برای نمونه، مشاهده افزایش زمان پاسخ تنها یک داده پایشی است؛ اما بررسی این موضوع که افزایش زمان پاسخ ناشی از جمع‌آوری زباله، انتظار روی قفل، عملیات فایل یا کندی شبکه بوده است، به داده‌های مشاهده‌پذیری دقیق‌تر نیاز دارد.

۳-۲-۲ پروفایل‌سازی

پروفایل‌سازی^۳ روشی برای اندازه‌گیری رفتار برنامه در زمان اجرا است. هدف آن شناسایی بخش‌هایی از برنامه است که بیشترین مصرف پردازنده، حافظه یا زمان اجرا را به خود اختصاص می‌دهند. داده‌های پروفایل می‌توانند در سطح تابع و متد، نخ، تخصیص حافظه، قفل، عملیات ورودی و خروجی یا فراخوانی سیستمی جمع‌آوری شوند.

دو رویکرد اصلی در پروفایل‌سازی عبارت‌اند از ابزارگذاری و نمونه‌برداری. در ابزارگذاری^۴، نقاط مشخصی از برنامه یا زمان اجرا برای ثبت ورود، خروج یا رخدادهای مورد نظر تغییر داده می‌شوند. این روش می‌تواند داده‌های دقیق و کاملی تولید کند، اما به دلیل اجرای منطق اضافی در تعداد زیادی از نقاط برنامه، احتمال ایجاد سربار بالا وجود دارد. در نمونه‌برداری^۵، وضعیت برنامه در فاصله‌های زمانی مشخص بررسی می‌شود. این روش معمولاً سربار کمتری دارد، ولی ممکن است رخدادهای کوتاه‌مدت میان دو نمونه را ثبت نکند.

²Observability

³Profiling

⁴Instrumentation

⁵Sampling

۴-۲-۲ رخدادهای معیار و گزارش

در یک سامانه پایش، داده‌ها را می‌توان در سه سطح در نظر گرفت:

- **رخداد:** یک اتفاق منفرد در زمان مشخص، مانند آغاز جمع‌آوری زباله، انتظار یک نخ روی قفل یا انجام یک عملیات خواندن فایل؛
- **معیار:** یک مقدار عددی قابل اندازه‌گیری، مانند تعداد رخدادها، مجموع زمان انتظار، میانگین مدت عملیات یا حجم داده انتقال یافته؛
- **گزارش:** نمایش ساخت‌یافته‌ای از رخدادها و معیارها برای تحلیل، مقایسه و تصمیم‌گیری.

رخدادهای خام جزئیات زیادی دارند، اما حجم آن‌ها بالا است و تحلیل مستقیم تک‌تک آن‌ها دشوار خواهد بود. به همین دلیل، معمولاً رخدادها در بازه‌های زمانی مشخص تجمیع می‌شوند تا معیارهایی مانند تعداد، مجموع، میانگین و بیشینه از آن‌ها به دست آید. این تبدیل، پایه اصلی پردازش جریانی در سامانه مورد بررسی است.

۳-۲ اجرای برنامه‌های جاوا و ماشین مجازی جاوا

برنامه‌های جاوا پس از ترجمه، به بایت‌کد تبدیل می‌شوند و بایت‌کد توسط ماشین مجازی جاوا^۶ اجرا می‌گردد. ماشین مجازی میان برنامه و سیستم‌عامل قرار دارد و وظایفی مانند بارگذاری کلاس‌ها، مدیریت حافظه، اجرای نخ‌ها، کامپایل درجا و جمع‌آوری زباله را انجام می‌دهد. این لایه میانی باعث قابلیت حمل برنامه‌های جاوا می‌شود، اما هم‌زمان رفتار زمان اجرای برنامه را پیچیده‌تر می‌کند.

۱-۳-۲ کامپایل درجا

ماشین مجازی جاوا می‌تواند بخش‌هایی از بایت‌کد را در زمان اجرا به کد ماشین تبدیل کند. این فرایند که کامپایل درجا^۷ نامیده می‌شود، بر اساس رفتار واقعی برنامه تصمیم می‌گیرد کدام متدها بیشتر اجرا شده‌اند و باید بهینه شوند. در نتیجه، عملکرد یک متد ممکن است در ابتدای اجرای برنامه با عملکرد همان متد پس از گرم‌شدن ماشین مجازی متفاوت باشد. این موضوع باید هنگام اندازه‌گیری عملکرد و طراحی آزمایش‌ها در نظر گرفته شود.

^۶Java Virtual Machine

^۷Just-In-Time Compilation

۲-۳-۲ مدیریت حافظه و جمع‌آوری زباله

در جاوا، حافظه اشیای برنامه معمولاً به صورت خودکار توسط جمع‌آورنده زباله مدیریت می‌شود. ایجاد تعداد زیاد شیء، نگهداری ناخواسته مراجع یا تنظیم نامناسب حافظه می‌تواند باعث افزایش دفعات یا مدت توقف‌های جمع‌آوری زباله شود. بنابراین، برای تحلیل مشکلات حافظه تنها مشاهده مقدار مصرف حافظه کافی نیست و باید رخدادهای تخصیص، وضعیت هیپ و فعالیت جمع‌آورنده زباله نیز بررسی شوند.

۳-۳-۲ نخ‌ها و هم‌زمانی

ماشین مجازی جاوا امکان اجرای هم‌زمان چند نخ را فراهم می‌کند. نخ‌ها ممکن است در حالت اجرا، آماده اجرا، انتظار، خواب یا مسدود روی قفل باشند. افزایش تعداد نخ‌ها همیشه به افزایش کارایی منجر نمی‌شود؛ زیرا رقابت بر سر پردازنده، قفل‌ها و منابع ورودی و خروجی می‌تواند زمان انتظار را افزایش دهد. مشاهده وضعیت نخ‌ها و نمونه‌های پشته اجرا برای تشخیص چنین مشکلاتی اهمیت دارد.

۴-۲ Java Flight Recorder

Java Flight Recorder یا JFR یک زیرسامانه ثبت رخداد در سکوی جاوا است که برای جمع‌آوری داده‌های زمان اجرا با سربار محدود طراحی شده است. این ابزار رخدادهای تولیدشده توسط ماشین مجازی، کتابخانه‌های جاوا و برنامه را در قالبی ساخت‌یافته ثبت می‌کند [۵].

۱-۴-۲ مدل رخداد در JFR

هر رخداد JFR معمولاً دارای نوع، زمان شروع، مدت، نخ مرتبط و مجموعه‌ای از فیلدهای اختصاصی است. برای نمونه، یک رخداد عملیات فایل می‌تواند شامل مسیر فایل، تعداد بایت، مدت عملیات و نخ اجراکننده باشد. یک رخداد قفل نیز ممکن است نام کلاس قفل، مدت انتظار و نخ مسدودشده را ثبت کند. ساخت‌یافته بودن این داده‌ها باعث می‌شود بتوان آن‌ها را بدون پردازش گزارش‌های متنی به مدل داده‌ای یکسان تبدیل کرد.

رخدادهای JFR را می‌توان از نظر مدت به دو گروه کلی تقسیم کرد. برخی رخدادها لحظه‌ای هستند و تنها وقوع یک اتفاق را نشان می‌دهند؛ برخی دیگر دارای زمان آغاز و مدت هستند. رخدادهای دارای مدت برای تحلیل تأخیر و شناسایی عملیات کند اهمیت بیشتری دارند.

۲-۴-۲ دسته‌های مهم رخدادهای JFR

JFR مجموعه گسترده‌ای از رخدادها را ارائه می‌کند که مهم‌ترین دسته‌های آن برای پایش عملکرد عبارت‌اند از:

- **پردازنده و اجرای متدها:** نمونه‌های اجرای نخ و نمونه‌های پشته که برای تشخیص مسیرهای پرتکرار و بخش‌های پرمصرف برنامه مفید هستند؛
- **حافظه:** رخدادهای تخصیص شیء، وضعیت هیپ و اطلاعات مرتبط با فشار حافظه؛
- **جمع‌آوری زباله:** زمان آغاز و پایان، علت اجرا، مدت توقف و وضعیت حافظه پیش و پس از جمع‌آوری؛
- **نخ و قفل:** آغاز و پایان نخ، توقف، پارک‌شدن، انتظار و رقابت روی ناظرها؛
- **فایل و شبکه:** عملیات خواندن و نوشتن، حجم داده و مدت عملیات؛
- **ماشین مجازی:** بارگذاری کلاس، کامپایل درجا و سایر رخدادهای داخلی زمان اجرا.

۳-۴-۲ ثبت پیوسته و مصرف جریانی

JFR می‌تواند داده‌ها را در یک فایل ثبت کند یا رخدادهای آن را در زمان اجرا در اختیار مصرف‌کننده قرار دهد. ثبت فایل برای تحلیل پس از رخداد مناسب است؛ اما در یک سامانه پایش بی‌درنگ، مصرف جریانی اهمیت بیشتری دارد. در این حالت، مصرف‌کننده به فرایند هدف متصل می‌شود، رخدادهای فعال را دریافت می‌کند و آن‌ها را به مسیر پردازش بعدی می‌فرستد.

مصرف جریانی امکان تحلیل نزدیک به زمان واقعی را فراهم می‌کند، اما باید نرخ تولید رخداد و هزینه پردازش آن‌ها کنترل شود. فعال کردن تعداد زیادی رخداد با آستانه بسیار پایین ممکن است خود ابزار پایش را به منبع سربار تبدیل کند. به همین دلیل، انتخاب نوع رخداد، دوره نمونه‌برداری و آستانه مدت باید متناسب با هدف پایش باشد.

۴-۴-۲ مزایای JFR

مهم‌ترین مزایای JFR عبارت‌اند از:

- یکپارچگی با ماشین مجازی و عدم نیاز به تغییر کد برنامه برای بسیاری از رخدادها؛
- ارائه داده‌های ساخت‌یافته درباره حافظه، نخ‌ها، قفل‌ها، فایل، شبکه و فعالیت‌های داخلی ماشین مجازی؛
- امکان تنظیم نوع رخداد، دوره نمونه‌برداری و آستانه ثبت؛
- سربار کمتر نسبت به ابزارگذاری کامل ورود و خروج همه متدها؛
- امکان ثبت فایل و نیز دریافت جریانی رخدادها.

۵-۴-۲ محدودیت‌های JFR

با وجود قابلیت‌های گسترده، JFR تمام رفتار سیستم را مشاهده نمی‌کند. این ابزار عمدتاً دیدی از درون ماشین مجازی و کتابخانه‌های جاوا ارائه می‌دهد. رخدادهایی که در سطح هسته رخ می‌دهند یا مستقیماً توسط برنامه‌های بومی ایجاد می‌شوند ممکن است به‌طور کامل در داده‌های JFR نمایان نباشند. همچنین، پروفایل‌سازی نمونه‌ای تضمین نمی‌کند که همه فراخوانی‌های کوتاه‌مدت ثبت شوند. انتساب دقیق مصرف منابع سیستم‌عامل به یک متد جاوا نیز تنها با داده‌های JFR همیشه امکان‌پذیر نیست.

۵-۲ فناوری eBPF و مشاهده رخدادهای سطح هسته

eBPF فناوری‌ای در هسته لینوکس است که امکان اجرای برنامه‌های کوچک و کنترل‌شده را در پاسخ به رخدادهای سیستم فراهم می‌کند. این برنامه‌ها می‌توانند به نقاط مختلف هسته یا فضای کاربر متصل شوند و داده‌های مورد نیاز را بدون تغییر کد منبع برنامه هدف جمع‌آوری کنند [۴].

۱-۵-۲ نحوه اجرای برنامه‌های eBPF

برنامه eBPF ابتدا به بایت‌کد مخصوص تبدیل می‌شود و سپس به هسته بارگذاری می‌گردد. پیش از اجرا، بررسی‌کننده^۸ هسته برنامه را تحلیل می‌کند تا از ایمنی دسترسی به حافظه، پایان‌پذیری مسیرهای اجرا و رعایت محدودیت‌های محیط هسته اطمینان حاصل کند. پس از تأیید، برنامه می‌تواند به یک نقطه اتصال متصل شود و هنگام وقوع رخداد اجرا گردد.

^۸Verifier

این مدل با اجرای یک برنامه عادی در فضای کاربر تفاوت دارد. برنامه eBPF نباید عملیات دلخواه و نامحدود انجام دهد؛ بلکه باید کوتاه، قابل بررسی و مناسب اجرای پرتکرار باشد. پردازش سنگین معمولاً در فضای کاربر انجام می‌شود و برنامه هسته تنها داده ضروری را استخراج و ارسال می‌کند.

۲-۵-۲ نقاط اتصال

فناوری eBPF از نقاط اتصال مختلفی پشتیبانی می‌کند. چند نمونه مهم عبارت‌اند از:

- **ردیاب‌های ایستا:** نقاط از پیش تعریف‌شده در هسته که برای مشاهده رخدادهایی مانند زمان‌بندی، فراخوانی سیستمی و شبکه استفاده می‌شوند؛
- **ردیاب‌های پویا:** اتصال به ورود یا خروج توابع هسته؛
- **ردیاب‌های فضای کاربر:** اتصال به توابع کتابخانه‌ها یا برنامه‌های فضای کاربر؛
- **نقاط شبکه:** اتصال به مسیر دریافت و ارسال بسته‌ها در لایه‌های مختلف شبکه.

انتخاب نقطه اتصال به نوع داده مورد نیاز بستگی دارد. برای نمونه، مشاهده عملیات فایل را می‌توان در سطح فراخوانی سیستمی یا توابع داخلی سامانه فایل انجام داد. هرچه نقطه اتصال به لایه پایین‌تری نزدیک باشد، جزئیات سیستم‌عامل بیشتر می‌شود، اما تفسیر داده و ارتباط آن با منطق برنامه دشوارتر خواهد بود.

۳-۵-۲ نقشه‌ها

نقشه‌های eBPF^۹ ساختارهای داده‌ای هستند که برای ذخیره و تبادل اطلاعات میان برنامه هسته و برنامه فضای کاربر استفاده می‌شوند. نقشه‌ها می‌توانند برای نگهداری شمارنده‌ها، وضعیت موقت رخداد، تنظیمات برنامه یا ارتباط دادن زمان ورود و خروج یک عملیات به کار روند. انواع مختلف نقشه برای الگوهای دسترسی متفاوت وجود دارد؛ برای مثال، نگاشت کلید به مقدار، آرایه و نگاشت ویژه هر پردازنده.

^۹eBPF Maps

۴-۵-۲ بافر حلقوی

برای ارسال تعداد زیادی رخداد از هسته به فضای کاربر می‌توان از بافر حلقوی^{۱۰} استفاده کرد. برنامه eBPF فضایی را در بافر رزرو می‌کند، داده رخداد را در آن می‌نویسد و سپس رخداد را برای مصرف‌کننده فضای کاربر منتشر می‌سازد. مصرف‌کننده داده‌ها را خوانده و به مرحله بعدی خط لوله منتقل می‌کند. ظرفیت بافر محدود است. اگر نرخ تولید رخداد از نرخ مصرف بیشتر شود، امکان از دست رفتن داده وجود دارد. بنابراین، کوچک نگه‌داشتن ساختار رخداد، کاهش پردازش در مسیر هسته و مصرف سریع داده‌ها برای پایداری سامانه ضروری است.

۵-۵-۲ شناسه فرایند و نخ

رخداد‌های سطح هسته معمولاً شامل شناسه فرایند و شناسه نخ هستند. این شناسه‌ها برای فیلترکردن رخداد‌های مربوط به برنامه هدف و هم‌بسته‌سازی داده‌های هسته با داده‌های ماشین مجازی اهمیت دارند. با این حال، شناسه نخ سیستم‌عامل همیشه به شکل مستقیم در همه رخداد‌های سطح برنامه نمایش داده نمی‌شود و ممکن است برای ایجاد ارتباط میان دو منبع داده به نگاشت یا اطلاعات تکمیلی نیاز باشد.

۶-۵-۲ مزایای eBPF

مزایای اصلی eBPF در پایش عملکرد عبارت‌اند از:

- مشاهده رخدادها در سطح سیستم‌عامل بدون تغییر کد برنامه هدف؛
- امکان فیلترکردن داده در محل تولید و کاهش حجم انتقال؛
- دسترسی به اطلاعات فراخوانی‌های سیستمی، فایل، شبکه و زمان‌بندی؛
- قابلیت اتصال پویا به نقاط مختلف هسته و فضای کاربر؛
- مناسب‌بودن برای ساخت ابزارهای پایش کم‌سربار و قابل توسعه.

¹⁰Ring Buffer

۷-۵-۲ محدودیت‌ها و چالش‌های eBPF

استفاده از eBPF به هسته لینوکس و امکانات نسخه مورد استفاده وابسته است. بارگذاری برنامه‌ها معمولاً به مجوزهای ویژه نیاز دارد و خطا در طراحی نقطه اتصال یا فیلتر می‌تواند حجم بسیار زیادی از رخداد تولید کند. همچنین، داده‌های سطح هسته از مفاهیم سطح برنامه مانند نام متد یا شیء جاوا آگاهی ندارند. از این رو، مشاهده اینکه یک نخ عملیات نوشتن فایل انجام داده است ساده‌تر از تعیین دقیق متد جاوایی مسئول آن عملیات است.

۶-۲ ترکیب داده‌های JVM و سیستم‌عامل

داده‌های JFR و eBPF دو دید مکمل از اجرای برنامه فراهم می‌کنند. JFR اطلاعات غنی‌تری از ساختار ماشین مجازی، نخ‌ها، پشته اجرا و حافظه جاوا ارائه می‌دهد. در مقابل، eBPF رفتار واقعی فرایند را در مرز سیستم‌عامل مشاهده می‌کند. ترکیب این دو منبع می‌تواند در تشخیص علت مشکلات عملکردی مؤثر باشد.

برای ترکیب رخدادها، وجود چند ویژگی مشترک ضروری است:

- **زمان رخداد:** همه رخدادها باید دارای مهر زمانی قابل مقایسه باشند؛
- **شناسه فرایند:** رخداد باید به فرایند جاوای هدف نسبت داده شود؛
- **شناسه نخ:** در صورت امکان، رخدادهای دو منبع باید بر اساس نخ مرتبط شوند؛
- **نوع و دسته رخداد:** رخدادهای ناهمگون باید در دسته‌های مشترکی مانند پردازنده، حافظه، فایل و شبکه قرار گیرند؛
- **قالب داده مشترک:** نام فیلدها، واحدها و ساختار پیام باید برای پردازش مراحل بعدی یکنواخت شود.

هم‌بستگی زمانی همواره به معنی رابطه علت و معلولی قطعی نیست. وقوع یک رخداد فایل در سطح هسته هم‌زمان با اجرای یک متد جاوا می‌تواند نشانه ارتباط باشد، اما برای اثبات انتساب دقیق ممکن است اطلاعات پشته، شناسه نخ و نقاط اتصال دقیق‌تری لازم باشد. بنابراین، سامانه‌های ترکیبی باید میان دقت انتساب، حجم داده و سربار پایش تعادل برقرار کنند.

۷-۲ انتقال و پردازش جریانی داده با Apache Kafka

سامانه‌های پایش زمان اجرا معمولاً با جریانی پیوسته از رخدادها روبه‌رو هستند. این رخدادها ممکن است با نرخ بالا و از چند منبع مختلف تولید شوند و لازم است بدون ایجاد وابستگی مستقیم میان جمع‌آورنده، پردازشگر و تحلیل‌گر منتقل شوند. استفاده از یک زیرساخت جریانی، امکان جداسازی این مؤلفه‌ها، ذخیره موقت داده‌ها و پردازش مستقل آن‌ها را فراهم می‌کند.

Apache Kafka یک سکوی توزیع‌شده برای انتشار، ذخیره و مصرف جریان‌های داده است. در این سکو، تولیدکنندگان پیام‌ها را در موضوع‌ها^{۱۱} منتشر می‌کنند و مصرف‌کنندگان می‌توانند آن‌ها را با سرعت و منطق پردازشی مستقل بخوانند [۱]. این جداسازی باعث می‌شود جمع‌آورنده رخداد تنها مسئول تولید داده باشد و نیازی به آگاهی از نحوه تجمیع، تحلیل یا گزارش‌گیری نداشته باشد.

هر موضوع در Kafka می‌تواند به چند پارتیشن تقسیم شود. پارتیشن‌بندی امکان توزیع ذخیره‌سازی و پردازش را فراهم می‌کند و ترتیب پیام‌ها را در محدوده هر پارتیشن حفظ می‌نماید. معمولاً کلید پیام تعیین می‌کند که پیام در کدام پارتیشن قرار گیرد. در داده‌های پایشی، کلید می‌تواند بر اساس شناسه فرایند، نخ، میزبان، نوع رخداد یا یک ویژگی منطقی دیگر انتخاب شود تا رخدادهای مرتبط در مسیر پردازشی مشترکی قرار گیرند.

پیام‌های منتشرشده در Kafka بلافاصله پس از مصرف حذف نمی‌شوند، بلکه بر اساس سیاست نگهداری برای مدت مشخصی باقی می‌مانند. این ویژگی امکان بازخوانی داده، پردازش مجدد رخدادها و افزودن مصرف‌کننده‌های جدید را فراهم می‌سازد. با این حال، مدت نگهداری باید متناسب با حجم داده و نیازهای تحلیل انتخاب شود؛ زیرا رخدادهای خام پایشی می‌توانند در مدت کوتاهی فضای ذخیره‌سازی زیادی مصرف کنند.

برای پردازش داده‌های موجود در Kafka می‌توان از Kafka Streams استفاده کرد. Kafka Streams یک کتابخانه جاوا برای ساخت برنامه‌های پردازش جریانی است که داده را از موضوع‌های ورودی می‌خواند، عملیات تبدیل، فیلتر، گروه‌بندی و تجمیع را انجام می‌دهد و نتیجه را در موضوع‌های خروجی منتشر می‌کند [۲]. این کتابخانه به‌صورت مستقیم با مدل داده و پارتیشن‌بندی Kafka یکپارچه است و برای ساخت پردازشگرهای سبک و توزیع‌پذیر مناسب است.

یکی از کاربردهای اصلی پردازش جریانی در سامانه‌های پایش، تبدیل رخدادهای خام به معیارهای قابل تحلیل است. رخدادها می‌توانند بر اساس یک کلید مشترک گروه‌بندی و در پنجره‌های زمانی

¹¹Topic

مشخص تجمیع شوند. برای هر پنجره می‌توان شاخص‌هایی مانند تعداد رخداد، مجموع زمان، میانگین، بیشینه، نرخ عملیات یا حجم داده را محاسبه کرد. این فرایند حجم داده را کاهش می‌دهد و خروجی مناسب‌تری برای تحلیل ناهنجاری فراهم می‌سازد.

در مجموع، Kafka و Kafka Streams دو نقش مکمل در خط لوله داده دارند. Kafka مسئول انتقال، نگهداری و توزیع جریان پیام‌ها است و Kafka Streams منطق پردازش، گروه‌بندی و تجمیع آن‌ها را اجرا می‌کند. ترکیب این دو فناوری، معماری‌ای ایجاد می‌کند که در آن جمع‌آوری داده، پردازش جریانی و تحلیل نهایی از یکدیگر جدا هستند و هر بخش می‌تواند به صورت مستقل توسعه و مقیاس داده شود. مزایای اصلی این رویکرد عبارت‌اند از:

- جداسازی تولیدکنندگان رخداد از مصرف‌کنندگان و تحلیل‌گران؛
- امکان پردازش هم‌زمان و توزیع‌شده داده‌ها بر اساس پارتیشن‌ها؛
- قابلیت بازخوانی و پردازش مجدد پیام‌ها؛
- پشتیبانی از عملیات پنجره‌ای و حالت‌دار؛
- امکان افزودن مسیرهای پردازشی جدید بدون تغییر جمع‌آورنده‌ها.

در مقابل، استفاده از این زیرساخت نیازمند مدیریت مناسب تعداد پارتیشن‌ها، سیاست نگهداری، حجم وضعیت‌های محلی، رخدادهای دیررس و ظرفیت ذخیره‌سازی است. همچنین، میزان موازی‌سازی پردازش به نحوه پارتیشن‌بندی داده و تعداد پارتیشن‌های ورودی وابسته خواهد بود.

۸-۲ تشخیص ناهنجاری در داده‌های پایشی

ناهنجاری^{۱۲} مشاهده‌ای است که با رفتار عادی یا مورد انتظار سامانه تفاوت معنادار دارد. در داده‌های پایشی، ناهنجاری می‌تواند به شکل افزایش ناگهانی مصرف پردازنده، رشد پیوسته حافظه، افزایش مدت عملیات فایل، کاهش نرخ شبکه یا افزایش زمان انتظار روی قفل ظاهر شود. تشخیص ناهنجاری با تشخیص خطا یکسان نیست. هر ناهنجاری الزاماً خطا نیست؛ برای نمونه، افزایش موقت پردازنده ممکن است ناشی از یک بار کاری معتبر باشد. همچنین، برخی خطاها ممکن

¹²Anomaly

است بدون تغییر واضح در معیارهای کلی رخ دهند. بنابراین، خروجی آشکارساز باید به‌عنوان نشانه‌ای برای بررسی بیشتر در نظر گرفته شود.

۱-۸-۲ روش آستانه‌ای

در روش آستانه‌ای، مقدار یک معیار با حد از پیش تعیین‌شده مقایسه می‌شود. اگر مقدار از آستانه عبور کند، وضعیت غیرعادی گزارش می‌شود. این روش ساده، سریع و قابل توضیح است. برای معیارهایی که محدوده قابل قبول مشخص دارند، مانند درصد پردازنده یا مدت انتظار، روش آستانه‌ای انتخاب مناسبی است.

محدودیت اصلی این روش ثابت‌بودن آستانه است. رفتار عادی یک برنامه ممکن است در ساعت‌های مختلف، بارهای متفاوت یا سخت‌افزارهای مختلف تغییر کند. آستانه بسیار پایین باعث هشدار کاذب و آستانه بسیار بالا باعث از دست رفتن ناهنجاری می‌شود.

۲-۸-۲ خط پایه غلتان

در روش خط پایه غلتان^{۱۳}، مقدار فعلی با خلاصه‌ای از چند پنجره قبلی مقایسه می‌شود. میانگین، میانه، انحراف معیار یا دامنه تغییرات می‌توانند برای ساخت خط پایه استفاده شوند. این روش خود را با رفتار اخیر برنامه تطبیق می‌دهد و برای معیارهایی که مقدار عادی ثابتی ندارند مناسب‌تر است. با این حال، اگر رفتار غیرعادی برای مدت طولانی ادامه پیدا کند، ممکن است به تدریج وارد خط پایه شود. همچنین، در شروع اجرای سامانه هنوز داده تاریخی کافی برای ساخت خط پایه وجود ندارد. این دوره معمولاً دوره گرم‌شدن تحلیل‌گر در نظر گرفته می‌شود.

۳-۸-۲ تشخیص روند

برخی مشکلات به‌صورت یک جهش ناگهانی ظاهر نمی‌شوند، بلکه در چند پنجره متوالی رشد می‌کنند. نشت حافظه نمونه‌ای از این وضعیت است. در روش تشخیص روند، جهت و شدت تغییر معیار در چند بازه متوالی بررسی می‌شود. افزایش پیوسته می‌تواند حتی پیش از عبور مقدار از یک آستانه مطلق، به‌عنوان نشانه هشدار شناسایی شود.

¹³Rolling Baseline

۴-۸-۲ معیارهای ارزیابی آشکارساز

برای ارزیابی یک آشکارساز می‌توان از معیارهای زیر استفاده کرد:

- **مثبت درست:** ناهنجاری وجود داشته و سامانه آن را تشخیص داده است؛
- **مثبت کاذب:** سامانه وضعیت عادی را ناهنجار گزارش کرده است؛
- **منفی کاذب:** ناهنجاری وجود داشته ولی سامانه آن را تشخیص نداده است؛
- **تأخیر تشخیص:** فاصله میان آغاز ناهنجاری و تولید گزارش؛
- **قابلیت توضیح:** امکان مشخص کردن معیار، آستانه و دلیل ایجاد هشدار.

در یک پروژه مهندسی پایش، روش‌های ساده و قابل توضیح مزیت مهمی دارند. کاربر باید بداند هشدار بر اساس کدام معیار و در چه بازه زمانی ایجاد شده است. به همین دلیل، ترکیب قواعد آستانه‌ای، خط پایه غلتان و تحلیل روند می‌تواند نقطه شروع مناسبی برای یک سامانه توسعه‌پذیر باشد.

۹-۲ ملاحظات سربار و دقت

هر ابزار پایش بر سامانه هدف اثر می‌گذارد. این اثر می‌تواند شامل مصرف پردازنده، حافظه، پهنای باند شبکه و فضای ذخیره‌سازی باشد. هدف یک سامانه پایش عملی حذف کامل سربار نیست، زیرا جمع‌آوری داده بدون هزینه امکان‌پذیر نیست؛ بلکه باید میان دقت، جزئیات و هزینه تعادل ایجاد شود. عوامل زیر بر سربار اثر دارند:

- تعداد و نوع رخدادهای فعال؛
- نرخ نمونه‌برداری و آستانه مدت رخداد؛
- اندازه ساختار هر پیام؛
- تعداد نقاط اتصال eBPF؛
- نرخ انتشار پیام در Kafka؛
- تعداد کلیدها و پنجره‌های فعال در پردازشگر؛

- مدت نگهداری داده خام و تجمیع‌شده؛

- پیچیدگی قواعد تحلیل ناهنجاری.

کاهش نرخ نمونه‌برداری یا افزایش آستانه ثبت، سرشار را کم می‌کند ولی احتمال از دست رفتن رخدادهای کوتاه را افزایش می‌دهد. افزایش اندازه پنجره تعداد خروجی‌ها را کاهش می‌دهد، اما تأخیر تشخیص را بیشتر می‌کند. ثبت فیلدهای بیشتر تحلیل دقیق‌تری فراهم می‌کند، ولی حجم پیام و هزینه شبکه را افزایش می‌دهد. بنابراین، تنظیم سامانه باید متناسب با هدف آزمایش و محیط اجرا انجام شود.

۲-۱۰ جمع‌بندی

در این فصل مفاهیم و فناوری‌های پایه مورد نیاز برای درک سامانه بررسی شدند. پایش برای بررسی شاخص‌های شناخته‌شده به کار می‌رود، در حالی که مشاهده‌پذیری داده لازم برای تحلیل وضعیت‌های پیش‌بینی‌نشده را فراهم می‌کند. پروفایل‌سازی نیز رفتار برنامه را در زمان اجرا اندازه‌گیری می‌کند و می‌تواند مبتنی بر ابزارگذاری یا نمونه‌برداری باشد.

JFR دیدی ساخت‌یافته از رفتار درون ماشین مجازی جاوا، شامل نخ‌ها، حافظه، جمع‌آوری زباله، قفل‌ها و عملیات ورودی و خروجی ارائه می‌دهد. در مقابل، eBPF امکان مشاهده رخدادهای سطح هسته و سیستم‌عامل را فراهم می‌کند. این دو فناوری مکمل یکدیگر هستند، ولی ترکیب داده‌های آن‌ها به هماهنگ‌سازی زمان، شناسه فرایند، شناسه نخ و قالب داده مشترک نیاز دارد.

همچنین، تجمیع پنجره‌ای روشی برای تبدیل حجم زیاد رخدادهای خام به معیارهای قابل تحلیل است. Kafka و Kafka Streams در کنار یکدیگر، انتقال، نگهداری، کلیدگذاری، گروه‌بندی و تجمیع حالت‌دار جریان داده را فراهم می‌کنند. در نهایت، روش‌های آستانه‌ای، خط پایه غلتان و تشخیص روند می‌توانند برای شناسایی رفتارهای غیرعادی روی داده‌های تجمیع‌شده به کار روند. فصل بعد با تکیه بر این مبانی، معماری سامانه و جریان داده میان اجزای آن را تشریح می‌کند.

فصل سوم

معماری سامانه

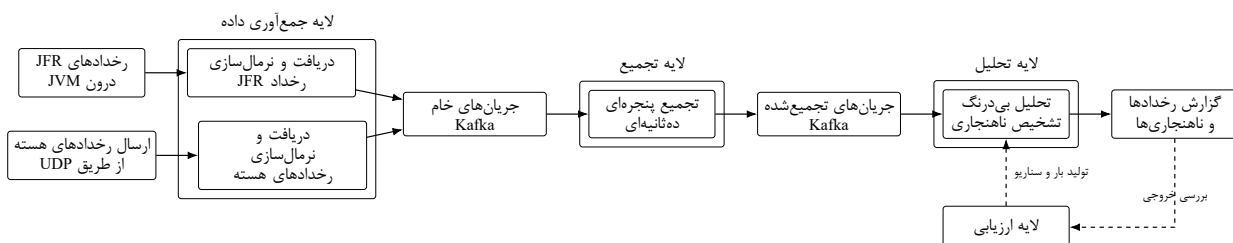
۱-۳ مقدمه

هدف اصلی سامانه، ثبت و پردازش رخدادهای مرتبط با کارایی فرایندهای JVM به گونه‌ای است که بتوان رفتار سامانه را به صورت بی‌درنگ تحلیل کرد. برای رسیدن به این هدف، معماری به صورت لایه‌ای و جریان‌محور طراحی شده است. در این معماری، رخدادها از منابع مختلف جمع‌آوری می‌شوند، به قالبی یکسان تبدیل می‌گردند، در بازه‌های زمانی کوتاه تجمیع می‌شوند و در نهایت برای تشخیص ناهنجاری مورد تحلیل قرار می‌گیرند.

نکته مهم در این فصل آن است که معماری سامانه در سطح مفهومی بررسی می‌شود. بنابراین تمرکز بر نقش هر لایه، جریان داده میان لایه‌ها و دلیل وجود هر بخش است، نه جزئیات پیاده‌سازی داخلی اجزا.

۲-۳ نمای کلی معماری

سامانه از چند لایه منطقی تشکیل شده است: لایه جمع‌آوری داده، لایه ذخیره‌سازی و انتقال جریان داده، لایه تجمیع، لایه تحلیل و لایه ارزیابی. لایه جمع‌آوری، رخدادها را از دو مسیر مستقل دریافت می‌کند. مسیر اول مربوط به رخدادهای JFR است که رفتار درون JVM را نشان می‌دهند. مسیر دوم مربوط به رخدادهای سطح هسته است که از بیرون فرایند و در سطح سیستم‌عامل مشاهده می‌شوند. پس از جمع‌آوری، هر دو مسیر داده به قالب مشترک تبدیل می‌شوند و در Kafka ذخیره می‌گردند. سپس لایه تجمیع، داده‌های خام را در پنجره‌های زمانی ده‌ثانیه‌ای خلاصه می‌کند و نتیجه را دوباره در Kafka قرار می‌دهد. در نهایت، لایه تحلیل داده‌های تجمیع‌شده را دریافت کرده و بررسی می‌کند که آیا نشانه‌ای از ناهنجاری عملکردی وجود دارد یا خیر.



شکل ۱-۳: نمای کلی معماری لایه‌ای سامانه

شکل ۱-۳ نمای کلی این معماری را نشان می‌دهد. همان‌طور که در شکل دیده می‌شود، مسیرهای

JFR و رخدادهای هسته در ابتدا جدا هستند، اما پس از نرمال سازی وارد مسیر مشترک ذخیره سازی، تجمیع و تحلیل می شوند.

۳-۳ مسیرهای داده در سامانه

در طراحی سامانه دو مسیر داده اصلی وجود دارد. این دو مسیر از نظر منبع تولید رخداد متفاوت اند، اما پس از نرمال سازی در یک مدل پردازشی مشترک قرار می گیرند.

۱-۳-۳ مسیر رخدادهای JFR

مسیر اول از رخدادهای JFR شروع می شود. این رخدادهای اطلاعاتی درباره رفتار درون JVM تولید می کنند؛ برای نمونه مصرف پردازنده، وضعیت حافظه، رخدادهای ورودی/خروجی فایل، ارتباطات شبکه، نخ ها و قفل ها. مولفه دریافت کننده JFR این رخدادهای را از فرایند هدف دریافت می کند و پیش از ذخیره سازی، آن ها را نرمال سازی می کند.

منظور از نرمال سازی در این مسیر، تبدیل رخدادهای خام JFR به ساختاری یکنواخت است؛ ساختاری که در آن منبع رخداد، نوع رخداد، دسته رخداد، زمان، مشخصات فرایند، اطلاعات نخ و داده های اختصاصی رخداد مشخص باشد. پس از این تبدیل، رخدادهای به عنوان داده خام در Kafka ذخیره می شوند تا لایه های بعدی بتوانند بدون وابستگی مستقیم به منبع JFR آن ها را پردازش کنند.

۲-۳-۳ مسیر رخدادهای هسته

مسیر دوم مربوط به رخدادهای سطح هسته یا kernel events است. این رخدادهای دیدی بیرونی تر نسبت به رفتار فرایند فراهم می کنند و برای مشاهده مواردی مانند رخدادهای فایل، شبکه و حافظه در سطح سیستم عامل مفید هستند. در این مسیر، رخدادهای هسته از طریق UDP به مولفه دریافت کننده رخدادهای هسته ارسال می شوند.

مولفه دریافت کننده هسته پس از دریافت پیام ها، آن ها را به مدل مشترک سامانه تبدیل می کند. در این مرحله، فیلدهایی مانند نوع رخداد، شناسه فرایند، شناسه نخ، زمان رخداد و دسته عملکردی استخراج یا تکمیل می شوند. سپس رخدادهای نرمال شده در مسیر خام Kafka ذخیره می شوند. به این ترتیب، هرچند منبع رخدادهای هسته با رخدادهای JFR متفاوت است، اما خروجی این لایه از نظر ساختاری با مسیر اول هم راستا می شود.

۴-۳ لایه جمع‌آوری داده

لایه جمع‌آوری داده مسئول تبدیل داده‌های خام ناهمگون به رخدادهای ساخت‌یافته و قابل پردازش است. این لایه دو وظیفه اصلی دارد: نخست، اتصال به منبع رخداد و دریافت داده؛ دوم، نرمال‌سازی رخدادهای و انتشار آن‌ها در جریان داده.

جداسازی مسیر JFR و مسیر رخدادهای هسته در این لایه یک تصمیم معماری مهم است. دلیل این جداسازی آن است که هر منبع داده ویژگی‌ها، نرخ تولید، روش اتصال و سطح مشاهده متفاوتی دارد. با این حال، هر دو مسیر پس از نرمال‌سازی به مدل مشترک سامانه می‌رسند. این کار باعث می‌شود لایه‌های بعدی، یعنی تجمیع و تحلیل، نیازی به شناخت جزئیات منبع اولیه نداشته باشند. در این لایه داده هنوز در سطح رخداد خام قرار دارد. بنابراین حجم داده می‌تواند زیاد باشد و نگهداری بلندمدت آن هزینه ذخیره‌سازی بالایی ایجاد کند. به همین دلیل، معماری سامانه یک لایه تجمیع جداگانه در ادامه مسیر قرار می‌دهد تا داده خام به معیارهای خلاصه‌تر تبدیل شود.

۵-۳ لایه انتقال و ذخیره‌سازی جریان داده

Kafka در این معماری نقش ستون فقرات انتقال داده را دارد. هر لایه، خروجی خود را در یک جریان مشخص منتشر می‌کند و لایه بعدی آن جریان را مصرف می‌کند. این طراحی چند مزیت دارد. نخست، اجزا از نظر اجرایی از یکدیگر جدا می‌شوند و توقف یا کندی یک بخش الزاماً باعث توقف کامل سامانه نمی‌شود. دوم، امکان کنترل نگهداری داده در هر مرحله وجود دارد. سوم، افزودن مصرف‌کننده‌های جدید، مانند داشبورد یا گزارش‌گیر مستقل، بدون تغییر در لایه جمع‌آوری امکان‌پذیر می‌شود.

در سطح معماری، دو نوع جریان در Kafka قابل تشخیص است: جریان‌های خام و جریان‌های تجمیع‌شده. جریان خام شامل رخدادهای نرمال‌شده‌ای است که مستقیماً از منابع JFR یا هسته به دست آمده‌اند. جریان تجمیع‌شده شامل خروجی پنجره‌های زمانی است و برای تحلیل و گزارش‌گیری مناسب‌تر است.

۶-۳ لایه تجمیع

لایه تجمیع، رخدادهای خام را به معیارهای قابل تحلیل تبدیل می‌کند. در این سامانه، تجمیع در پنجره‌های زمانی ده‌ثانیه‌ای انجام می‌شود. برای هر پنجره، رخدادهای بر اساس کلیدهایی مانند شناسه

فرایند، منبع رخداد، دسته رخداد و نوع رخداد گروه‌بندی می‌شوند و سپس معیارهایی مانند تعداد رخداد، مجموع مدت، میانگین مدت، بیشینه مدت و معیارهای اختصاصی هر دسته محاسبه می‌گردد. وجود این لایه از چند جهت اهمیت دارد. نخست، تحلیل ناهنجاری معمولاً بر اساس روندها و معیارهای خلاصه‌شده معنادارتر از تحلیل تک‌تک رخدادهای خام است. دوم، تجمیع باعث کاهش حجم داده‌ای می‌شود که باید برای مدت طولانی‌تر نگهداری شود. در نتیجه، لازم نیست برای لایه جمع‌آوری داده، زمان نگهداری یا retention زیادی در نظر گرفته شود. داده خام می‌تواند با زمان نگهداری کوتاه‌تر ذخیره شود و داده تجمیع‌شده، که حجم کم‌تر و ارزش تحلیلی بیش‌تری دارد، برای تحلیل و گزارش‌گیری حفظ گردد. این تصمیم باعث کاهش مصرف دیسک و ساده‌تر شدن مدیریت داده می‌شود. سوم، لایه تجمیع مرزی میان مشاهده رخداد و تحلیل رفتاری ایجاد می‌کند. لایه جمع‌آوری صرفاً رخداد را ثبت می‌کند، اما لایه تجمیع آن رخدادها را به شاخص‌هایی تبدیل می‌کند که برای تصمیم‌گیری قابل استفاده‌اند.

۷-۳ لایه تحلیل

لایه تحلیل، مصرف‌کننده داده‌های تجمیع‌شده است. این لایه خروجی پنجره‌های زمانی را دریافت می‌کند و بررسی می‌کند که آیا معیارهای مشاهده‌شده نشانه‌ای از رفتار غیرعادی دارند یا خیر. از آنجا که ورودی این لایه داده خام نیست، تحلیل‌گر با معیارهایی کار می‌کند که از قبل خلاصه، دسته‌بندی و زمان‌بندی شده‌اند.

در این معماری، تحلیل ناهنجاری می‌تواند برای دسته‌های مختلف رخداد به صورت جداگانه انجام شود؛ برای نمونه پردازنده، حافظه، ورودی/خروجی فایل، شبکه، قفل‌ها و رخدادهای هسته. این جداسازی باعث می‌شود منطق تشخیص برای هر دسته متناسب با ماهیت همان دسته تعریف شود. برای مثال، معیار مناسب برای فشار پردازنده با معیار مناسب برای ازدحام قفل یا فشار حافظه یکسان نیست. خروجی لایه تحلیل، گزارش رخدادهای تجمیع‌شده و گزارش ناهنجاری‌ها است. گزارش ناهنجاری نشان می‌دهد در چه بازه زمانی، برای چه دسته‌ای از رخدادها و بر اساس کدام معیار، رفتار غیرعادی مشاهده شده است.

۳-۸ لایه ارزیابی

در کنار لایه‌های اصلی سامانه، یک لایه ارزیابی نیز وجود دارد. این لایه بخشی از مسیر عادی پایش نیست، بلکه برای سنجش توان سامانه استفاده می‌شود. نقش آن ایجاد سناریوهای کنترل‌شده و بررسی این موضوع است که آیا لایه تحلیل می‌تواند ناهنجاری مورد انتظار را تشخیص دهد یا خیر. لایه ارزیابی سناریوهایی مانند افزایش مصرف پردازنده، فشار حافظه و جمع‌آوری زباله، ورودی/خروجی سنگین فایل، ترافیک شبکه، رقابت روی قفل‌ها و سناریوهای وابسته به رخدادهای هسته را ایجاد می‌کند. سپس خروجی تحلیل‌گر بررسی می‌شود و گزارشی شامل وضعیت تشخیص، معیار تشخیص‌داده‌شده و تأخیر تشخیص تولید می‌گردد.

۳-۹ مدل داده مشترک

یکی از نکات کلیدی معماری، تعریف مدل داده مشترک پس از لایه جمع‌آوری است. هر رخداد، صرف نظر از این که از JFR آمده باشد یا از مسیر هسته، باید اطلاعات پایه‌ای مشابهی داشته باشد. این اطلاعات شامل منبع رخداد، نوع رخداد، دسته رخداد، زمان، شناسه فرایند و داده‌های تکمیلی است. وجود مدل مشترک باعث می‌شود لایه تجمیع بتواند هر دو مسیر داده را با منطق یکسان پردازش کند. همچنین لایه تحلیل می‌تواند بر اساس دسته و معیار تصمیم بگیرد، نه بر اساس جزئیات منبع تولید رخداد. این انتزاع یکی از دلایل توسعه‌پذیری معماری است؛ زیرا با اضافه شدن منبع داده جدید، کافی است خروجی آن منبع به مدل مشترک تبدیل شود.

۳-۱۰ دلایل انتخاب معماری لایه‌ای

انتخاب معماری لایه‌ای برای این سامانه چند دلیل اصلی دارد:

- **جداسازی مسئولیت‌ها:** هر لایه یک وظیفه مشخص دارد؛ جمع‌آوری، انتقال، تجمیع، تحلیل یا ارزیابی.
- **کاهش وابستگی اجزا:** ارتباط اجزا از طریق جریان‌های Kafka انجام می‌شود و هر بخش می‌تواند مستقل‌تر توسعه یا اجرا شود.

- **مدیریت بهتر حجم داده:** داده خام با نگهداری کوتاه‌تر ذخیره می‌شود و داده تجمیع‌شده برای تحلیل و گزارش حفظ می‌گردد.
- **پشتیبانی از چند منبع مشاهده:** معماری می‌تواند هم رخدادهای درون JVM و هم رخدادهای سطح هسته را در یک مدل تحلیلی مشترک قرار دهد.
- **امکان توسعه آینده:** افزودن منبع داده، معیار تجمیع یا قانون تحلیل جدید بدون تغییر بنیادین در کل سامانه امکان‌پذیر است.

۱۱-۳ جمع‌بندی

در این فصل معماری سامانه از دید لایه‌ای بررسی شد. سامانه دو مسیر داده اصلی دارد: مسیر رخدادهای JFR و مسیر رخدادهای هسته. هر دو مسیر در لایه جمع‌آوری نرمال‌سازی شده و در Kafka ذخیره می‌شوند. سپس لایه تجمیع، داده‌های خام را در پنجره‌های ده‌ثانیه‌ای به معیارهای خلاصه تبدیل می‌کند. لایه تحلیل این معیارها را برای تشخیص ناهنجاری مصرف می‌کند و لایه ارزیابی نیز با ایجاد سناریوهای کنترل‌شده، توان سامانه را می‌سنجد. این معماری با جداسازی مسئولیت‌ها و استفاده از مدل داده مشترک، امکان تحلیل بی‌درنگ و توسعه‌پذیر رخدادهای کارایی فرایندهای JVM را فراهم می‌کند.

فصل چهارم

پیاده‌سازی سامانه

۱-۴ ساختار مخزن

مخزن پروژه یک پروژه چندماژوله است. فایل اصلی pom.xml ماژول‌های event-receiver، kernel-event-receiver، event-aggregator و profiler-evaluation را تعریف می‌کند و نسخه جاوا را ۱۷ قرار می‌دهد. بخش event-analytics با پایتون پیاده‌سازی شده و وابستگی‌های آن در فایل requirements.txt آمده است. بخش kernel-event-sender نیز شامل کد eBPF، بارگذار C و Makefile است.

۲-۴ دریافت رخدادهای JFR

ورودی اصلی رخدادهای سطح برنامه در کلاس aut.ce.App از ماژول event-receiver آغاز می‌شود. این کلاس نام ماشین مجازی هدف را از متغیر محیطی TARGET_VM_NAME می‌خواند و در صورت نبود مقدار، نام JfrEventGenerator را در نظر می‌گیرد. سپس JfrRemoteListener و JfrRemoteEventStreamer اجرا می‌شوند و نخ KafkaWriter پیام‌ها را به Kafka ارسال می‌کند. برای استخراج فیلدهای تخصصی، الگوی غنی‌ساز رخداد استفاده شده است. هر غنی‌ساز بخشی از بار رخداد را می‌سازد و خروجی نهایی به صورت پیام ساخت‌یافته ارسال می‌شود. این روش باعث می‌شود اضافه کردن یک نوع رخداد جدید با افزودن یک غنی‌ساز جدید انجام شود.

۳-۴ دریافت رخدادهای هسته

بخش kernel-event-sender شامل برنامه eBPF و بارگذار کاربرفضا است. برنامه eBPF رخدادهای مرتبط را از هسته دریافت می‌کند و بارگذار آن‌ها را به مقصد شبکه ارسال می‌کند. ماژول kernel-event-receiver این رخدادها را دریافت، بافر و در Kafka منتشر می‌کند. این مسیر برای سناریوهایی مفید است که نشانه اصلی آن‌ها در سطح سیستم‌عامل دیده می‌شود.

۴-۴ تجمیع رخدادها

کلاس EventAggregatorApp قلب ماژول تجمیع است. این کلاس دو توپولوژی مشابه می‌سازد: یکی برای رخدادهای JFR و دیگری برای رخدادهای هسته. رخدادها ابتدا فیلتر می‌شوند تا پیام‌های نامعتبر حذف شوند. سپس بر اساس کلید تجمیع گروه‌بندی شده و در پنجره‌های زمانی پردازش می‌شوند.

خروجی هر پنجره شامل زمان شروع و پایان، مدت پنجره، کلید و معیارهای محاسبه شده است. راهبردهای تجمیع در بسته strategy قرار دارند. این راهبردها تفاوت رخدادهای پردازنده، حافظه، فایل، شبکه و جمع‌آوری زباله را از منطق عمومی Kafka Streams جدا می‌کنند. در نتیجه، تغییر روش محاسبه یک معیار نیازمند تغییر توپولوژی اصلی نیست.

۴-۵ تحلیل و گزارش‌گیری

در ماژول event-analytics، فایل main.py پیکربندی را می‌خواند، مصرف‌کننده Kafka را می‌سازد، آشکارسازها را مقداردهی می‌کند و پیام‌های تجمیع‌شده را پردازش می‌کند. آشکارسازهای موجود شامل پردازنده، حافظه، ورودی/خروجی فایل، شبکه، قفل و هسته هستند. خروجی‌ها توسط CsvReportWriter در قالب گزارش رخدادهای تجمیع‌شده و گزارش ناهنجاری نوشته می‌شوند.

۴-۶ پایش خود سامانه

پروژه برای مشاهده وضعیت اجزای خود نیز از Prometheus استفاده می‌کند. در ماژول‌های جاوا، کلاس‌های MetricsHandler معیارهایی مانند مقدارهای تجمیع‌شده را به‌روزرسانی می‌کنند. پوشه monitoring شامل پیکربندی لازم برای اجرای Prometheus است [۶].

۴-۷ جمع‌بندی

پیاده‌سازی پروژه بر اصل جداسازی مسئولیت‌ها تکیه دارد. دریافت‌کننده‌ها مسئول تبدیل رخداد خام به پیام، تجمیع‌کننده مسئول تبدیل پیام‌ها به معیارهای پنجره‌ای، و تحلیل‌گر مسئول تصمیم‌گیری درباره ناهنجاری‌ها است.

فصل پنجم

ارزیابی، نتیجه‌گیری و کارهای آینده

۱-۵ روش ارزیابی

برای ارزیابی سامانه، ماژول profiler-evaluation سناریوهای کنترل‌شده‌ای را اجرا می‌کند. این ماژول یک فرایند جاوا راه‌اندازی می‌کند، شناسه فرایند را چاپ می‌کند و پس از یک تأخیر اولیه، سناریوها را اجرا می‌کند تا دریافت‌کننده‌های JFR و هسته فرصت اتصال داشته باشند. خروجی ارزیابی شامل فایل‌های experiment_results.csv، experiment_metrics.json و evaluation_report.md است.

۲-۵ سناریوهای آزمایش

سناریوهای پیاده‌سازی‌شده در پروژه عبارت‌اند از:

- CpuSpikeScenario برای ایجاد افزایش ناگهانی مصرف پردازنده؛
- MemoryGcPressureScenario برای ایجاد فشار حافظه و تحریک جمع‌آوری زباله؛
- FileIoBurstScenario برای تولید الگوی شدید ورودی/خروجی فایل؛
- LockContentionScenario برای ایجاد رقابت روی قفل‌ها؛
- NetworkBurstScenario برای ایجاد ترافیک شبکه؛
- MmapFileScanScenario برای بررسی رفتار دسترسی حافظه‌نگاشته به فایل.

هر سناریو یک رفتار هدفمند ایجاد می‌کند و سپس گزارش تحلیلی بررسی می‌شود تا مشخص شود آیا ناهنجاری مورد انتظار تشخیص داده شده است یا خیر. این روش برای پروژه حاضر مناسب است، زیرا امکان کنترل زمان شروع، مدت اجرا و نوع فشار ایجادشده را فراهم می‌کند.

۳-۵ معیارهای ارزیابی

دو معیار اصلی برای ارزیابی در نظر گرفته می‌شود. معیار نخست موفقیت تشخیص است؛ یعنی آیا سامانه برای سناریوی اجراشده، ناهنجاری مناسب را گزارش کرده است یا نه. معیار دوم تأخیر تشخیص است؛ یعنی فاصله زمانی بین شروع سناریو و ثبت ناهنجاری در گزارش. این دو معیار در کنار هم نشان می‌دهند سامانه هم از نظر دقت عملی و هم از نظر سرعت واکنش چه وضعیتی دارد.

۴-۵ بحث

طراحی خط لوله‌ای پروژه چند مزیت مهم دارد. نخست، هر جزء به صورت مستقل اجرا و عیب‌یابی می‌شود. دوم، استفاده از Kafka امکان جداسازی نرخ تولید و مصرف پیام را فراهم می‌کند. سوم، جمع‌آوری پنجره‌ای حجم داده خام را کاهش داده و تحلیل را ساده‌تر می‌کند. چهارم، ترکیب رخدادهای JFR و eBPF دید گسترده‌تری نسبت به رفتار برنامه ایجاد می‌کند.

در مقابل، سامانه محدودیت‌هایی نیز دارد. کیفیت تشخیص به انتخاب آستانه‌ها و طول پنجره وابسته است. همچنین اجرای بخش eBPF به محیط لینوکس و دسترسی‌های لازم نیاز دارد. از سوی دیگر، بسیاری از آزمون‌های موجود در پروژه حالت اسکلت اولیه دارند و برای استفاده پژوهشی کامل، لازم است پوشش آزمون واحد و آزمون یکپارچه افزایش یابد.

۵-۵ نتیجه‌گیری

در این پایان‌نامه، سامانه‌ای برای پایش عملکرد و تشخیص ناهنجاری در برنامه‌های جاوا بررسی، طراحی و مستندسازی شد. سامانه با استفاده از JFR رخدادهای سطح ماشین مجازی را دریافت می‌کند، با eBPF امکان مشاهده رخدادهای سطح هسته را فراهم می‌سازد، با Kafka Streams داده‌ها را در پنجره‌های زمانی جمع‌آوری می‌کند و با سرویس تحلیلی پایتونی ناهنجاری‌ها را گزارش می‌دهد. نتیجه اصلی این پژوهش آن است که ترکیب مشاهده‌پذیری سطح برنامه و سطح سیستم‌عامل در یک معماری جریانی، امکان تحلیل دقیق‌تر و توسعه‌پذیرتر رفتار عملکردی را فراهم می‌کند.

۶-۵ کارهای آینده

برای توسعه آینده پروژه، موارد زیر پیشنهاد می‌شود:

- افزودن داشبورد تصویری برای نمایش زنده معیارها و ناهنجاری‌ها؛
- استفاده از روش‌های یادگیری ماشین برای تنظیم پویا و داده‌محور آستانه‌ها؛
- افزایش پوشش آزمون‌های واحد و یکپارچه برای ماژول‌های جاوا و پایتون؛
- افزودن همبستگی بین رخدادهای JFR و رخدادهای هسته بر اساس زمان و شناسه فرایند؛

- پشتیبانی از استقرار کانتینری کامل برای اجرای ساده‌تر کل خط لوله؛
- افزودن معیارهای دقت، فراخوانی و نرخ هشدار کاذب به گزارش ارزیابی.

کتاب نامه

- [1] Apache Software Foundation. Apache kafka documentation. <https://kafka.apache.org/documentation/>, 2026. Accessed 2026-06-23.
- [2] Apache Software Foundation. Kafka streams documentation. <https://kafka.apache.org/documentation/streams/>, 2026. Accessed 2026-06-23.
- [3] CaOmDiEn. Ce-final-project: Jvm profiler and anomaly detection pipeline. <https://github.com/CaOmDiEn/CE-FINAL-PROJECT>, 2026. Commit b23e1b4bc1224cb0ea922dff86189ed8c3d5f437, accessed 2026-06-23.
- [4] eBPF Foundation. ebpf documentation. <https://ebpf.io/what-is-ebpf/>, 2026. Accessed 2026-06-23.
- [5] Oracle. Java flight recorder runtime guide. <https://docs.oracle.com/javacomponents/jmc-5-4/jfr-runtime-guide/about.htm>, 2026. Accessed 2026-06-23.
- [6] Prometheus Authors. Prometheus documentation. <https://prometheus.io/docs/introduction/overview/>, 2026. Accessed 2026-06-23.

پیوست اول

راهنمای اجرای پروژه

این پیوست خلاصه‌ای از روش اجرای خط لوله پروژه را ارائه می‌کند. دستورهای زیر بر اساس ساختار مخزن CE-FINAL-PROJECT نوشته شده‌اند.

آ-۱ ساخت ماژول‌های جاوا

```
mvn package -DskipTests
```

آ-۲ اجرای اجزای اصلی

ابتدا Kafka و سرویس‌های وابسته اجرا می‌شوند. سپس دریافت‌کننده رخدادهای JFR، دریافت‌کننده رخدادهای هسته، تجمیع‌کننده و تحلیل‌گر اجرا می‌شوند. اسکریپت `scripts/run-profiler-pipelines.sh` برای هماهنگ کردن این مراحل در مخزن پروژه قرار دارد.

آ-۳ اجرای ارزیابی

برای اجرای ماژول ارزیابی:

```
mvn -pl profiler-evaluation -am package -DskipTests
```

```
java -cp profiler-evaluation/target/classes aut.ce.evaluation.App
```

برنامه در شروع اجرا شناسه فرایند را چاپ می‌کند. برای بررسی رخدادهای هسته، فرستنده eBPF با همان شناسه فرایند ساخته و اجرا می‌شود:

```
cd kernel-event-sender
```

```
make clean
```

```
make TARGET_PID=12345
```

```
sudo ./build/loader
```

آ-۴ خروجی‌ها

خروجی‌های ارزیابی در قالب فایل‌های زیر تولید می‌شوند:

experiment_results.csv

experiment_metrics.json

evaluation_report.md

گزارش نهایی شامل وضعیت موفقیت تشخیص در هر سناریو و تأخیر تشخیص است.

واژه‌نامه‌ی فارسی به انگلیسی

Detection تشخیص	آ
خ	
Rolling Baseline خط پایه غلتان	آشکارسازی ناهنجاری . Anomaly Detection
د	آستانه Threshold
	ا
Event Receiver دریافت‌کننده رخداد	ارزیابی Evaluation
ر	
Event رخداد	ازدحام قفل Lock Contention
	ب
Kernel Event رخداد هسته	
ز	بازه زمانی Time Window
Runtime زمان اجرا	بافر Buffer
س	ت
Overhead سربار	تجمیع Aggregation

Observability نظارت‌پذیری	Experiment Scenario . . . سناریوی آزمایش
و	ش
File Input/Output ورودی/خروجی فایل	Process Identifier شناسه فرایند
	ص
	Message Queue صف پیام
	ف
	Memory Pressure فشار حافظه
	Target Process فرایند هدف
	ک
	Aggregation Key کلید تجمیع
	گی
	Anomaly Report گزارش ناهنجاری
	م
	Java Virtual Machine . . ماشین مجازی جاوا
	Metric معیار
	Topic موضوع
	ن
	Anomaly ناهنجاری

واژه‌نامه‌ی انگلیسی به فارسی

A	Experiment Scenario . . . سناریوی آزمایش
Aggregation تجمیع	F
Aggregation Key کلید تجمیع	File Input/Output ورودی/خروجی فایل
Anomaly ناهنجاری	J
Anomaly Detection . آشکارسازی ناهنجاری	Java Virtual Machine . . ماشین مجازی جاوا
Anomaly Report گزارش ناهنجاری	K
B	Kernel Event رخداد هسته
Buffer بافر	L
D	Lock Contention ازدحام قفل
Detection تشخیص	M
E	Memory Pressure فشار حافظه
Evaluation ارزیابی	Message Queue صف پیام
Event رخداد	Metric معیار
Event Receiver دریافت‌کننده رخداد	O
	Observability نظارت‌پذیری

Overhead	سربار	Runtime	زمان اجرا
P		T	
Process Identifier	شناسه فرایند	Target Process	فرایند هدف
R		Threshold	آستانه
Rolling Baseline	خط پایه غلتان	Time Window	بازه زمانی
		Topic	موضوع

Abstract

Modern software systems require continuous observability mechanisms that can capture performance symptoms with low overhead and transform them into actionable reports. This thesis presents the design and implementation of a streaming profiler pipeline for Java applications. The system combines Java Flight Recorder events, kernel-level events collected through eBPF, Apache Kafka for event transport, Kafka Streams for windowed aggregation, and a Python analytics service for anomaly detection.

The proposed architecture separates event collection, aggregation, detection, and evaluation into independent components. JFR and kernel receivers normalize raw observations into structured Kafka messages. The aggregation service groups events in configurable time windows and computes metrics for CPU usage, memory pressure, file I/O, network traffic, lock contention, garbage collection, and kernel counters. The analytics service applies threshold-based, rolling-baseline, and trend-based detectors and writes aggregate and anomaly reports in CSV format. An evaluation module executes controlled workload scenarios, including CPU spikes, memory and GC pressure, file I/O bursts, lock contention, network bursts, and memory-mapped file scans, then checks whether the pipeline reports the expected anomalies. The implementation demonstrates that a modular stream-processing design can combine JVM-level and operating-system-level telemetry while keeping each processing stage independently deployable and extensible.

Keywords: Performance monitoring, anomaly detection, Java Flight Recorder, eBPF, Kafka Streams, observability.



**Amirkabir University of Technology
(Tehran Polytechnic)**

Department of Computer Engineering

M. Sc. Thesis

**Design and Implementation of a
Performance Monitoring and Anomaly
Detection Pipeline for Java Applications
Using JFR, eBPF, and Kafka**

By

Student Name Student Surname

Supervisor

Dr. Supervisor Name

Advisor

Dr. Advisor Name

Month & Year